Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

1. Aim

To study the limitations of low-code development platforms when developing complex enterprise applications and to identify the challenges related to scalability, customization, security, integration, and performance.

2. Introduction

Low-code development platforms allow users to create applications using visual development tools, drag-and-drop components, predefined templates, and minimal manual coding. Platforms such as Zoho Creator, Microsoft Power Apps, and OutSystems help organizations develop applications quickly and reduce development effort.

However, complex enterprise applications often require advanced business logic, integration with multiple systems, high performance, strict security, and large-scale data processing. These requirements may exceed the capabilities or practical limits of a low-code platform.

3. Case Study Description

Case Study: Enterprise Resource Planning (ERP) System for a Large Manufacturing Company

Consider a large manufacturing company that wants to develop an ERP application to manage inventory, production,

employee information, suppliers, customer orders, billing, and financial transactions.

Initially, the company chooses a low-code platform to build the application quickly. Basic forms, reports, dashboards, and approval workflows are developed successfully. As the application grows, the company faces challenges in handling complex business rules, integrating external systems, managing large amounts of data, and meeting enterprise security requirements.

This case study examines the major limitations encountered and the possible solutions.

4. Limitations of Low-Code Platforms

4.1 Scalability Limitations

As the number of users and transactions increases, application performance may decrease. Platform-specific limits on records, execution time, or concurrent processing can affect large enterprise workloads.

Solution: Evaluate platform limits, optimize data access, and use specialized backend services where necessary.

4.2 Limited Customization

Prebuilt components may not support every specialized business requirement. Complex algorithms and advanced user-interface behavior may require custom code.

Solution: Use supported custom functions, APIs, extensions, or pro-code components.

4.3 Integration Challenges

Enterprise systems may need to connect with ERP software, CRM applications, payment gateways, databases, and legacy systems. Integration can become difficult when APIs are restricted or data formats differ.

Solution: Use APIs, middleware, integration services, and standardized data formats.

4.4 Security and Governance

Large enterprises require role-based access, data protection, audit trails, compliance controls, and centralized administration. Incorrect permission configuration can expose sensitive information.

Solution: Apply least-privilege access, configure roles carefully, monitor audit logs, and conduct regular security reviews.

4.5 Performance Limitations

Complex queries, repeated workflows, and large datasets can increase response times. Excessive automation may also create unnecessary processing.

Solution: Optimize queries, reduce redundant workflows, use appropriate data models, and test performance under realistic workloads.

4.6 Vendor Lock-in

Applications built using platform-specific components and scripting languages may be difficult to migrate to another platform.

Solution: Use standard APIs, document business logic, maintain data-export strategies, and assess portability before choosing a platform.

4.7 Testing and Maintenance

Frequent application changes may affect existing workflows and integrations. Testing every dependency can become difficult as the system grows.

Solution: Maintain documentation, use development and testing environments, perform regression testing, and establish change-management procedures.

5. Comparison of Challenges and Solutions

Limitation	Enterprise impact	Recommended solution
Scalability	Slow response during peak usage	Optimize architecture and offload heavy workloads
Customization	Difficult to implement specialized features	Use custom code or pro-code services
Integration	Data inconsistencies across systems	Use APIs and middleware
Security	Risk of unauthorized access	Enforce role-based permissions and audits
Performance	Delays in processing transactions	Tune queries and optimize workflows
Vendor lock-in	Difficult platform migration	Follow portable integration and data practices
Maintenance	Regression errors after updates	Use testing and change management

6. Findings

The case study identifies that low-code platforms are effective for rapid application development, prototypes, internal tools, and standard business workflows. However, complex

enterprise applications may require additional engineering when they involve high transaction volumes, specialized business logic, extensive integrations, or strict regulatory requirements.

A hybrid approach that combines low-code development with conventional programming and dedicated backend services can address many of these challenges.

7. Result

The case study on the limitations of low-code platforms for a complex enterprise use case was completed. The major challenges related to scalability, customization, integration, security, performance, vendor lock-in, and maintenance were identified, and suitable solutions were suggested.

Thus, the study concludes that low-code platforms can accelerate enterprise application development, but careful architectural planning and pro-code integration may be necessary to meet complex enterprise requirements.